Heaven's Light is Our Guide

Rajshahi University of Engineering and Technology



Course Code
ECE 3112

Course Title
Microprocessor, Assembly Language & Interfacing Sessional

Experiment Date: May 11, 2024,
Submission Date: May 25, 2024

Lab Report 5: Using Assembly Language to Program 8086 Microprocessor to Solve Problems.

Submitted to	**Submitted by**
Md. Omaer Faruq Goni	Md. Tajim An Noor
Lecturer	Roll: 2010025
Dept of ECE, Ruet	

Problem 4.1

Problem Description

Write a code to take input from user as Binary and output it as Binary.

Tools Used

- emu8086 - 8086 Microprocessor Emulator
- MacTeX -LaTeXcompiler.
- VS Code with LaTeXworkshop extension as a text editor.

Process

Code:

```
    —

DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    CNT DB 17
DATA ENDS

CODE SEGMENT
    ASSUME CS: CODE, DS: DATA

START:
    MOV AX, DATA
    MOV DS, AX

    XOR BX, BX
    MOV CX, 16

    MOV AH, 1
    INT 21H

INPUT:
    CMP AL, 0DH
    JE EXIT

    AND AL, 0FH
```

```
25      SHL BX, 1
26      OR BL, AL
27
28      INT 21H
29
30      JMP INPUT
31
32  EXIT:
33      MOV DL, CR
34      MOV AH, 2
35      INT 21H
36      MOV DL, LF
37      INT 21H
38
39  PRINT:
40      DEC CNT
41      JZ END_
42
43      RCL BX, 1
44      JC PRINT1
45      JNC PRINT0
```

```
46
47  PRINT1:
48      MOV DL, '1'
49      MOV AH, 2
50      INT 21H
51
52      JMP PRINT
53
54  PRINT0:
55      MOV DL, '0'
56      MOV AH, 2
57      INT 21H
58
59      JMP PRINT
60
61  END_:
62      MOV AH, 4CH
63      INT 21H
64
65  CODE ENDS
66  END START
```

Output

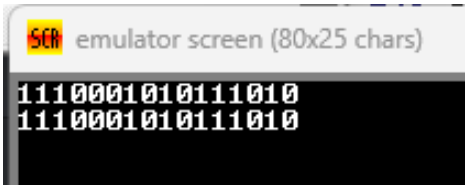


Figure 1: Binary input & output

Problem 4.2

Problem Description

Write a code to take input from user as Hexadecimal and output it as Hexadecimal.

Process

Code:

```
;
DATA SEGMENT
    CR EQU 0DH
    LF EQU 0AH
    CNT DB 4
DATA ENDS

CODE SEGMENT
    ASSUME CS: CODE, DS: DATA

START:
    MOV AX, DATA
    MOV DS, AX

    XOR BX, BX
    MOV CX, 4

    MOV AH, 1
    INT 21H

INPUT:
    CMP AL, 0DH
    JE EXIT

    CMP AL, 39H
    JG LETTER

    AND AL, 0FH
    JMP SHIFT

LETTER:
    SUB AL, 37H

SHIFT:
    SHL BX, CL
    OR BL, AL

    INT 21H

    JMP INPUT
```

```
40
41  EXIT:
42      MOV DL, CR
43      MOV AH, 2
44      INT 21H
45      MOV DL, LF
46      INT 21H
47
48  PRINT:
49      MOV DL, BH
50      SHR DL, 4
51      ROL BX, 4
52
53      CMP DL, 10
54      JL CONV_NUM
55      JGE CONV_CHAR
56
57  CONV_NUM:
58      OR DL, 30H
59      MOV AH, 2
60      INT 21H
61
62      LOOP PRINT
63
64  CONV_CHAR:
65      SUB DL, 9
66      OR DL, 40H
67      MOV AH, 2
68      INT 21H
69
70      LOOP PRINT
71
72  END1:
73      MOV AH, 4CH
74      INT 21H
75
76  CODE ENDS
77  ENDS START
```

Output

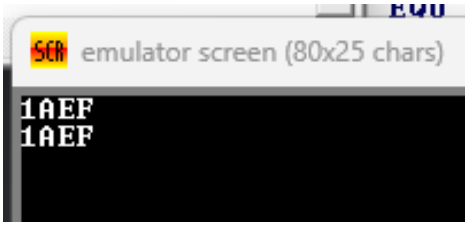


Figure 2: Hexadecimal input & output

Keywords

These are the new keywords learned while solving the Problems

EQU Used to declare a variable as a constant.

XOR Used to perform Exclusive-OR operation over each bit in a byte/word with the corresponding bit in another byte/word. Used here on the same variable to clear the register.

AND Used for adding each bit in a byte/word with the corresponding bit in another byte/word.

OR Used to multiply each bit in a byte/word with the corresponding bit in another byte/word.

JE Used to jump if equal/zero flag ZF = 1

SHL Used to shift bits of a byte/word towards left and put zero(S) in LSBs.

RCL Used to rotate bits of byte/word towards the left, i.e. MSB to CF and CF to LSB.

JC Used to jump if carry flag CF = 1

JNC Used to jump if no carry flag (CF = 0)

SHR Used to shift bits of a byte/word towards the right and put zero(S) in MSBs.

ROL Used to rotate bits of byte/word towards the left, i.e. MSB to LSB and to Carry Flag [CF].

JL Used to jump if less than/not greater than/equal instruction satisfies.

JGE Used to jump if greater than/equal/not less than instruction satisfies.